159.102 Notes

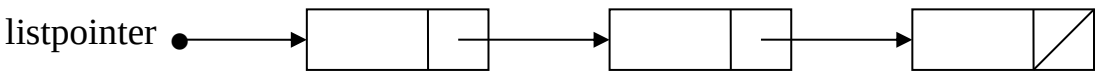
Section 11 : Linked lists

Linked Lists

A linked list is a data structure that differs from arrays in two important ways:

- it is always exactly the right length, i.e. no memory is wasted
- it allows only **sequential access**, i.e. to access an item in the linked list, you have to start at the beginning and move through every item in the list until you find the item you want

A linked list consists of **nodes**. Each node contains data and the address of the next node (a pointer). It is possible to set up nodes such that each node stores different data – unlike an array where every element has to be the same data type. However, in practice every node is usually the same data type. A linked list is usually represented like this:



The list consists of **nodes**. Each node has two sections: the **data** and the **next pointer**. There is a separate pointer called the **listpointer** that points to the first node in the list.

Program 1: set up a linked list with one node in the list

```
#include <iostream>
using namespace std;

class Node {
private:
  int value;
  // you can include more data items here
  Node *next;
public:
  void loaddata(int v, Node* p);
  int getvalue() { return value; }
  Node* getnext() { return next; }
  void setvalue(int v) { value = v; }
  void setnext(Node* p) { next = p; }
};

Node* createnode(int num);

Node* listpointer;

int main() {
  Node* firstnode;
  listpointer = NULL;
  firstnode = createnode(45);
  listpointer = firstnode;
}
```

```
Node* createnode(int num) {
  Node* temp;
  temp = new Node;
  temp->loaddata(num, NULL);
  return temp;
}

//---------- methods for the Node ---------
void Node::loaddata(int v, Node* p) {
  value = v;
  next = p;
}
```

Some important notes about Program 1:

- we make a class for a node, we do not make a class for a linked list
- you can have as many data items inside the node as you like (this node has only one integer value). Note the pointer called `next` that will hold the address of the next node in the list.
- there is one global variable called listpointer. Unlike an array where memory is set aside when the array is created, the linked list does not immediately set memory aside.
- the listpointer is set to NULL at the start of the program. This is a very good idea – just like setting a total to zero at the start of a progrm.
- the main program creates a new node and sets `firstnode` to point to the new node. Actually you could write the program without the variable `firstnode`.
- the code in the function createnode is very important. Set aside a block of memory (using new) and put the address into a temporary pointer. Then return the temporary pointer to main.
- IMPORTANT – the pointer of the last node in the list must be NULL. This is the only way that you know you have reached the end of the list. In this program, the function called createnode loads NULL into the next pointer.

Program 2: inserting several nodes at the FRONT of a linked list

```
#include <iostream>
using namespace std;

class Node {
private:
  int value;
  // you can include more data items here
  Node *next;
public:
  void loaddata(int v, Node* p);
  int getvalue() { return value; }
  Node* getnext() { return next; }
  void setvalue(int v) { value = v; }
  void setnext(Node* p) { next = p; }
};

void displaylist();

Node* listpointer;

int main() {
  Node *temp;
```

```
  int number;
  listpointer = NULL;
  number = 999;  // make sure number is > zero
  while (number >= 0) {
    cout << "Enter an integer value (-1 to stop) ";
    cin >> number;
    if (number >= 0) {
      temp = new Node;
      temp->loaddata(number, listpointer);
      listpointer = temp;
    }
  }
  displaylist();
}

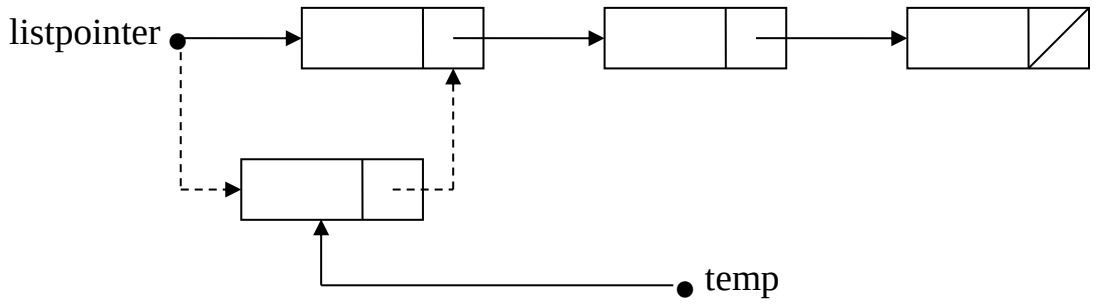
void displaylist() {
  Node* current;
  int currentvalue;
  current = listpointer;
  while (current != NULL) {
    currentvalue = current->getvalue();
    cout << currentvalue << ", ";
    current = current->getnext();
  }
}

//---------- methods for the Node ---------
void Node::loaddata(int v, Node* p) {
  value = v;
  next = p;
}
```

Some important notes about Program 2:

- we do not use a function called createnode. No real reason – you can do it with or without the function so this is an opportunity to look at creating nodes without using a function.
- it is REALLY, REALLY important that listpointer is set to NULL at the start of the program because when a new node is created, it's next pointer is set to listpointer. The first time this happens, listpointer is NULL so the first node (also initially the last node) has the next pointer set to NULL.
- the line number = 999 is a rather clunky way of setting up a loop with a trailing value of -1. You may think of a more elegant way of doing this. Each value is loaded into a new node.
- all other nodes are inserted BEFORE the first node. This done by making the next pointer of each new node point to the rest of the list (i.e. set next to listpointer). Then change listpointer to point to the new node (i.e. the new node becomes the first node in the list).
- the code in the function displaylist is very IMPORTANT. This is the code you need to "move along" a linked list. The pointer current is set to listpointer (i.e. current has the address of the first node). Then you can look at (or display) the value of the current node and then set current to be the next pointer in the current node. This makes current work through the list from one node to the next until it finds the node with the next pointer of NULL.
- when the program runs (make sure you try it out) the values are displayed in the reverse order to how they were entered. This is because each value is placed BEFORE the previous value.

When working with linked lists it is a really good idea (some programmers say it is absolutely vital) to draw a diagram of what is going on. Here is the diagram that describes Program 2:



Each time round the while loop we:

a) create a new node and set temp to point to the new node
b) set the next pointer of this new node to be equal to listpointer (i.e. the new node now points to the first node in the list)
c) set listpointer equal to temp (i.e. the new node is now the first node in the linked list)

HINT – when you want a pointer to point to a node, set it to be equal to a pointer that is already pointing to that node.

Program 3: inserting several nodes at the BACK of a linked list

```
#include <iostream>
using namespace std;

class Node {
private:
  int value;
  // you can include more data items here
  Node *next;
public:
  void loaddata(int v, Node* p);
  int getvalue() { return value; }
  Node* getnext() { return next; }
  void setvalue(int v) { value = v; }
  void setnext(Node* p) { next = p; }
};

void displaylist();

Node* listpointer;

int main() {
  Node *temp, *lastnode;
  int number;
  listpointer = NULL;
  // set up the first node
  cout << "Enter the first integer value ";
  cin >> number;
  listpointer = new Node;
  listpointer->loaddata(number, NULL);
  lastnode = listpointer;
  // insert the other nodes
```

```
  while (number >= 0) {
    cout << "Enter an integer value (-1 to stop) ";
    cin >> number;
    if (number >= 0) {
      temp = new Node;
      temp->loaddata(number, NULL);
      lastnode->setnext(temp);
      lastnode = temp;
    }
  }
  displaylist();
}

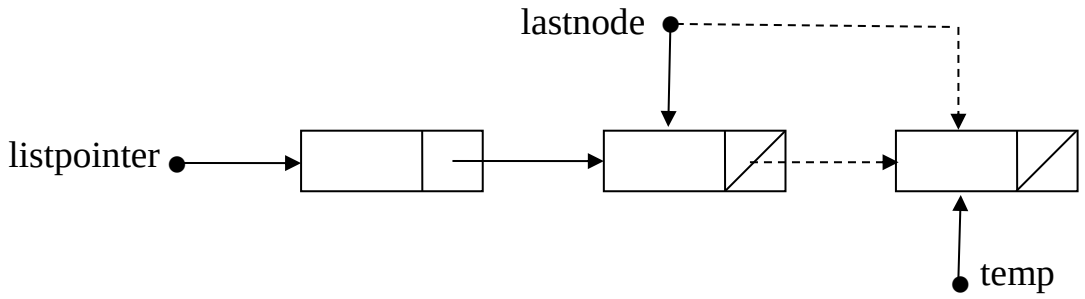
void displaylist() {
  Node* current;
  int currentvalue;
  current = listpointer;
  while (current != NULL) {
    currentvalue = current->getvalue();
    cout << currentvalue << ", ";
    current = current->getnext();
  }
}

//---------- methods for the Node ---------
void Node::loaddata(int v, Node* p) {
  value = v;
  next = p;
}
```

Some important notes about Program 3:

- the class and the function displaylist are exactly the same as Program 2
- listpointer does not really need to be set to NULL in this program because it immediately gets set to point to the first node. However, it is good to be in the habit of setting it to NULL.
- a pointer called lastnode is always set to point to the last node in the linked list
- the lastnode next pointer is changed to point to the new node (i.e. the new node is at the back)
- when the program runs (make sure you try it out) the values are displayed in the same order as they were entered. This is because each value is placed AFTER the previous value.

When working with linked lists it is a really good idea (some programmers say it is absolutely vital) to draw a diagram of what is going on. Here is the diagram that describes Program 3:



Each time round the while loop we:

a) create a new node and set temp to point to the new node

b) set the next pointer of this new node to be equal to NULL (i.e. this new node always becomes the last node in the list)
c) set the next pointer of the last node to be equal to temp (i.e. the node that used to be the last node now points to the new node, thus making the new node the last node)
d) set lastnode equal to temp (i.e. lastnode now points to the new last node)

Program 4: deleting a node from a linked list

```
#include <iostream>
using namespace std;

class Node {
private:
  int value;
  // you can include more data items here
  Node *next;
public:
  void loaddata(int v, Node* p);
  int getvalue() { return value; }
  Node* getnext() { return next; }
  void setvalue(int v) { value = v; }
  void setnext(Node* p) { next = p; }
};

void displaylist();

Node* listpointer;

int main() {
  Node *temp, *temp2, *previous;
  int i;
  listpointer = NULL;
  // insert three nodes in the list
  for (i = 0; i < 3; i++) {
    temp = new Node;
    temp->loaddata(i * 10, listpointer);
    listpointer = temp;
  }
  displaylist();
  cout << endl;
  // now remove the second node
  temp = listpointer->getnext();  // the second node
  temp2 = temp->getnext();
  previous = listpointer;
  previous->setnext(temp2);
  delete temp;  // delete the node to free memory
  displaylist();
  cout << endl;
}

void displaylist() {
  Node* current;
  int currentvalue;
```

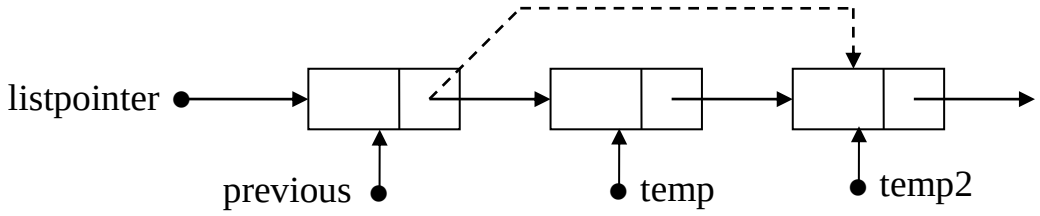
```
  current = listpointer;
  while (current != NULL) {
    currentvalue = current->getvalue();
    cout << currentvalue << ", ";
    current = current->getnext();
  }
}

//---------- methods for the Node ---------
void Node::loaddata(int v, Node* p) {
  value = v;
  next = p;
}
```

Some important notes about Program 4:

- the class and the function displaylist are exactly the same as Program 3
- the program sets up a linked list of 3 nodes and displays the list
- the program removes the second node and displays the list again
- the second node is not actually deleted – the pointers in the linked list are changed to avoid this particular node
- use `delete` to completely remove an unwanted node from memory

When working with linked lists it is a really good idea (some programmers say it is absolutely vital) to draw a diagram of what is going on. Here is the diagram that describes Program 4:



We use the following algorithm:

a) decide which node must be deleted
b) starting at the listpointer, work through the list until temp is pointing at the node to be deleted
c) set the pointer previous to point to the node just before temp and temp2 to point to the node just after temp (i.e. temp2 is set to the next pointer of temp)
d) set the next pointer of previous to be temp2 (i.e. when working through the linked list, the node that comes after previous will now be temp2 (i.e. temp is no longer in the list)
e) delete the node pointed to by temp

Advantages and Disadvantages of linked lists

Advantages of linked lists

- Memory is never wasted. When a new node is created, the correct amount of memory is set aside. When a node is removed (and the programmer uses delete) the memory is released for general use **
- No need to guess what size a linked list needs to be – it is always exactly the right size
- Each node could store different data types (unlike arrays)

Disadvantages of linked lists

- Way more complicated to use than arrays and easy to make a mistake

- Sequential access – the only way to use a linked list is to start at the listpointer and work your way through the list to the node that you want. Working sequentially through a very long linked list can significantly reduce the execution speed of a program.

** note: the process of allocating memory only when it is required is called **dynamic memory allocation**. The process of allocating a fixed block of memory before using it (as we do for arrays) is called **static memory allocation**.

And now for the good news about linked lists

Linked lists are available in the STL library collection. You don't have to create your own nodes as we have done in the programs above – you can rely on the library to do this for you.

Note: you do need to know how to create and use your own nodes for 159.102 – as we have done in the programs above. However if you are writing an application that needs a linked list, you could choose to use the library instead of developing your own code to create and manipulate nodes.

Program 5: using the linked list library

```
#include <iostream>
#include <list>
using namespace std;

void displaylist(list<int> l);

list<int> mylist;

int main() {
  int value;
  value = 999;  // ensure that value > zero
  while (value >= 0) {
    cout << "Enter an integer value (-1 to stop) ";
    cin >> value;
    if (value >= 0) {
      mylist.push_back(value);
    }
  }
  displaylist(mylist);
}

void displaylist(list<int> l) {
  list<int>::iterator ptr;
  for (ptr = l.begin(); ptr != l.end(); ptr++) {
    cout << *ptr << ", ";
  }
  cout << endl;
}
```

Important points about this program:

- it includes the `list` library
- it creates a list of integer values but you could create a list of any data type or class
- you have to specify the data type `<int>` when creating the list

- lists have several methods such as push_back which is used to insert a data item at the back
- the function displaylist uses something called an **iterator**
- an iterator is a pointer that can point to any item in the specified type of list
- begin is a method that returns a pointer (iterator) to the first item in the list
- end is a method that returns a pointer that is past the end of the list (note that end is used with 'not equal to')
- iterators use ++ which does not mean "add 1", it means "move on to the next node in the list"
- note that the iterator is a pointer and so the '*' is used to refer to the contents of the pointer

The list class

The list class provides you with variables and methods. You should know the basic methods associated with a list from the library. These are:

`iterator` – a pointer that can point to any item in a list

`begin()` – returns an iterator pointing to the first item in the list
`end()` – returns an iterator just past the last item in the list
`push_back()` – insert a new item at the back of the list
`push_front()` – insert a new item at the front of the list
`pop_back()` – delete the item at the back of the list
`pop_front()` – delete the item at the front of the list
`size()` – the number of items in the list

There are many other methods available with lists but you do not need to know them for our course. You may use any of them if you want to. The full description of the list class is here (if you are interested in a method, select it in the left-hand side bar):
https://www.cplusplus.com/reference/list/list/push_back/